

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Zlatko Pračić

API ZA RAD S RAZLIČITIM FORMATIMA DATOTEKA APACHE POI

**SEMINAR
NAPREDNE WEB TEHNOLOGIJE I SERVISI**

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Zlatko Pračić

Matični broj: 0016145097

Studij: Baze podataka i baze znanja

Kolegij: Napredne Web tehnologije i servisi

API ZA RAD S RAZLIČITIM FORMATIMA DATOTEKA APACHE POI

SEMINAR

Mentor:

Prof. dr. sc. Dragutin Kermek

Varaždin, kolovoz 2026.

Izjava o izvornosti

Izjavljujem kako je ovaj seminar izvorni rezultat mog rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Opsega od 100 do 300 riječi. Sažetak upućuje na temu rada, ukratko se iznosi čime se rad bavi, teorijsko-metodološka polazišta, glavne teze i smjer rada te zaključci.

Ključne riječi: riječ; riječ; riječ; Obuhvaća 7 ± 2 ključna pojma koji su glavni predmet rasprave u radu.

Sadržaj

1. Uvod	1
2. Pregled Apache POI projekta	2
2.1. Povijest i razvoj	2
2.2. Arhitektura i komponente	2
2.3. Pregled dostupnih klasa	3
3. Rad s formatima datoteka	4
3.1. Excel datoteke	4
3.2. Word dokumenti	4
3.3. PowerPoint prezentacije	5
4. Praktični primjeri	6
4.1. Kreiranje Word dokumenta	6
4.1.1. Zaglavlje i podnožje	6
4.1.2. Paragrafi i stilovi	7
4.1.3. Tablica	7
4.1.4. Spremanje dokumenta	8
4.1.5. Izrada cirkularnog dokumenta	9
4.2. Generiranje Excel izvještaja	11
5. Zaključak	12
Popis isječaka koda	13
Popis literature	14
Popis slika	15
Popis tablica	16

1. Uvod

U suvremenom okruženju razmjena podataka putem Microsoft Office dokumenata svakodnevna je pojava. Excel tablice, Word dokumenti i PowerPoint prezentacije postali su de facto standard za izvještavanje, analizu podataka i komunikaciju. Međutim, ručna obrada takvih dokumenata u sustavima koji zahtijevaju automatizaciju, poput sustava za upravljanje sadržajem, poslovnih aplikacija ili alata za ekstrakciju podataka, nije praktična ni skalabilna.

Apache POI projekt, koji razvija i održava Apache Software Foundation, nudi rješenje za ovaj problem u obliku besplatne Java biblioteke otvorenog koda. Biblioteka programerima omogućuje čitanje, pisanje i izmjenu Microsoft Office datoteka isključivo korištenjem Java koda, bez potrebe za instaliranim Microsoft Officeom. Time se postiže potpuna platformska neovisnost, što je posebno vrijedno u enterprise okruženjima koja se oslanjaju na Java ekosustav.

Motivacija za odabir Apache POI-a kao teme ovog rada leži upravo u njegovoj rasprostranjenosti i praktičnoj primjenjivosti. Biblioteka je jedna od najkorištenijih Java biblioteka za rad s Office formatima i aktivno se razvija već više od dva desetljeća. Podržava kako starije binarne formate temeljene na OLE2 standardu (XLS, DOC, PPT), tako i moderne XML-bazirane OOXML formate uvedene s Microsoft Officeom 2007 (XLSX, DOCX, PPTX).

Cilj ovog rada je sustavno prikazati arhitekturu i ključne komponente Apache POI projekta te demonstrirati njegovu primjenu na konkretnim primjerima. Rad je strukturiran na sljedeći način: drugo poglavlje donosi pregled povijesti projekta i njegovih tehničkih komponenti, treće poglavlje opisuje rad s pojedinim formatima datoteka, a četvrto poglavlje prikazuje praktične primjere generiranja Excel izvještaja i obrade Word dokumenta. Na kraju se iznosi zaključak s osvrtom na prednosti, ograničenja i smjernice za primjenu biblioteke.

2. Pregled Apache POI projekta

2.1. Povijest i razvoj

Začeci Apache POI projekta sežu u travanj 2001. godine, kada je programer Andrew Oliver dobio kratkoročni ugovor za generiranje Excel izvještaja u Javi. Suočen s drastičnim porastom cijene komercijalne biblioteke koju je dotad koristio, Oliver je odlučio razviti vlastito rješenje otvorenog koda. Uz pomoć Marca Johnsona, kojeg je pronašao putem lokalne Java korisničke grupe, implementirana je podrška za OLE 2 *Compound Document Format* kao temelj cijelog projekta. Johnson je preuzeo razvoj niskorazinskog sloja za rukovanje OLE2 strukturom, dok je Oliver paralelno radio na parsiranju XLS formata. Već s prvim izdanjem, POI 1.0, projekt je nadišao izvorne planove, uz podršku za čitanje i pisanje Excel datoteka, dodan je i serijalizacijski okvir za integraciju s drugim sustavima [1].

Kako je projekt rastao, postalo je jasno da nadilazi okvire jednog namjenskog alata. Nakon neuspjelog pokušaja integracije isključivo u Apache Cocoon projekt, donesena je odluka da se jezgri POI API-ji prenesu u okvir Apache Jakarta projekta, dok su pojedine komponente donirane drugim projektima iz Apache ekosustava. U sklopu Jakarte, projekt je nastavio rasti zahvaljujući doprinosima zajednice, proširujući podršku na nove formate i slučajeve korištenja. Početkom 2007. godine POI je zaprovodio proces inkubacije i postao samostalni projekt najviše razine unutar *Apache Software Foundation* [1].

Naziv projekta izvorno je bio akronim za “*Poor Obfuscation Implementation*”, humoristična referenca na činjenicu kako su Microsoftovi binarni formati, unatoč naizglednoj namjernoj kompliciranosti, bili uspješno reverzno razvijeni. Ista logika imenovanja proteže se i na potprojeke poput HSSF (*Horrible SpreadSheet Format*), HWPF (*Horrible Word Processor Format*) i ostale. Kako je projekt počeo privlačiti poslovne korisnike, objašnjenja akronima tiho su uklonjena s službenih stranica [2].

2.2. Arhitektura i komponente

Na ovoj infrastrukturi izgrađeni su moduli koji programerima nude intuitivan API za rad s Office dokumentima, apstrahirajući složenost internih binarnih i XML struktura. Pregled svih modula, njihovih formata i standarda prikazan je u Tablici 1, a detaljniji pregled ključnih klasa po kategorijama dan je u nastavku [3].

Tablica 1: Pregled Apache POI modula

Modul	Format	Ekstenzija	Standard
POIFS	OLE2 datotečni sustav	—	OLE2
HSSF	Excel (stari)	.xls	OLE2
XSSF	Excel (novi)	.xlsx	OOXML
SXSSF	Excel (streaming)	.xlsx	OOXML
HWPF	Word (stari)	.doc	OLE2
XWPF	Word (novi)	.docx	OOXML
HSLF	PowerPoint (stari)	.ppt	OLE2
XSLF	PowerPoint (novi)	.pptx	OOXML
HSMF	Outlook poruke	.msg	OLE2
HDGF/XDGF	Visio dijagrami	.vsd/.vsdx	OLE2/OOXML
HPBF	Publisher	.pub	OLE2

2.3. Pregled dostupnih klasa

Apache POI biblioteka pruža iznimno bogat skup klasa koje pokrivaju gotovo sve aspekte rada s Office dokumentima. Za Excel datoteke, klase poput `XSSFWorkbook`, `XSSFSheet`, `XSSFCell` i `XSSFCellStyle` omogućuju kreiranje i formatiranje tablica, dok `XSSFChart` i `XDDFChartData` podržavaju generiranje grafova. Za streaming obradu velikih datoteka dostupne su `SXSSFWorkbook` i `SXSSFSheet`. Word dokumenti pokriveni su klasama `XWPFDocument`, `XWPFParagraph`, `XWPFRun`, `XWPFTable` i `XWPFHeaderFooter`, a za PowerPoint prezentacije koriste se `XMLSlideShow`, `XSLFSlide`, `XSLFTextShape` i `XSLFTable` [4].

Uz rad s dokumentima, biblioteka nudi i infrastrukturne komponente. `POIFSFileSystem` i `OPCPackage` upravljaju niskorazinskim pristupom OLE2 i OOXML datotečnim strukturama. Klase iz paketa `EncryptionInfo`, `Encryptor` i `Decryptor` omogućuju enkripciju i dekripciju dokumenata. Za ekstrakciju teksta dostupne su `XWPFWordExtractor`, `ExcelExtractor` i `SlideShowExtractor`, a `DataFormatter` i `DateUtil` pomažu pri interpretaciji formata ćelija. Klase poput `EmbeddedExtractor` i `PictureData` omogućuju ekstrakciju ugrađenih slika i objekata iz dokumenata [4].

3. Rad s formatima datoteka

Ovo poglavlje rada posvećeno je praktičnoj primjeni Apache POI biblioteke za rad s tri najzastupljenija Microsoft Office formata — Excel tablicama, Word dokumentima i PowerPoint prezentacijama. Za svaki format postoje dvije implementacije koje odgovaraju dvjema generacijama Office standarda: starija binarna implementacija temeljena na OLE2 formatu te novija implementacija temeljena na OOXML standardu. U nastavku su opisane ključne mogućnosti svake implementacije, uz osvrt na specifičnosti pojedinih slučajeva korištenja [3].

3.1. Excel datoteke

Za rad s Excel datotekama Apache POI nudi tri implementacije. HSSF (*Horrible Spreadsheet Format*) pokriva stariji binarni .xls format koji odgovara Excel verzijama 97–2003, a XSSF (*XML Spreadsheet Format*) namijenjen je modernom .xlsx formatu temeljenom na OOXML standardu. Oba modula podržavaju čitanje i pisanje, a dostupna je i zajednička SS apstrakcija koja omogućuje pisanje koda neovisnog o konkretnom formatu. Isti kod može generirati i .xls i .xlsx datoteku promjenom jednog parametra. Kroz ove module moguće je kreirati radne knjige i listove, postavljati vrijednosti ćelija, primjenjivati stilove i formate, koristiti formule te upravljati uvjetnim oblikovanjem [5].

Za slučajeve kada je potrebno generirati izrazito velike Excel datoteke, standardni XSSF pristup može biti problematičan zbog visokog memorijskog opterećenja, cijela struktura dokumenta drži se u memoriji. Kao rješenje, od verzije 3.8-beta3 dostupan je SXSSF (*Streaming Spreadsheet Format*), streaming implementacija izgrađena povrh XSSF-a. SXSSF postiže mali memorijski otisak ograničavanjem pristupa samo redovima unutar kliznog prozora fiksne veličine, dok se stariji redovi automatski zapisuju na disk. Veličina prozora može se konfigurirati pri inicijalizaciji radne knjige ili zasebno po listu. Važno je napomenuti kako je SXSSF namijenjen isključivo pisanju, za čitanje velikih datoteka preporučuje se korištenje SAX-temeljenog Event modela koji XML strukturu .xlsx datoteke procesira inkrementalno [5].

3.2. Word dokumenti

Za rad s Word dokumentima Apache POI nudi dvije implementacije. HWPf (*Horrible Word Processor Format*) pokriva stariji binarni .doc format koji odgovara Word verzijama 97–2003, a XWPf (*XML Word Processor Format*) namijenjen je modernom .docx formatu temeljenom na OOXML standardu. HWPf podržava čitanje i ograničeno pisanje, a pruža i jednostavnu ekstrakciju teksta za još starije Word 6 i Word 95 formate. XWPf nudi stabilniji i potpuniji API te podržava čitanje i pisanje složenijih dokumenata [6].

Glavna ulazna točka za rad s .docx datotekama je klasa XWPfDocument, iz koje se dohvaćaju svi elementi dokumenta. Paragrafi su dostupni kroz klasu XWPfParagraph, a svaki paragraf sastoji se od jednog ili više XWPfRun objekata koji predstavljaju tekstualne odsječke s jednakim stilskim svojstvima: fontom, veličinom, podebljavanjem, nakošenje slova

i slično. Tablice se dohvaćaju kroz klasu `XWPFTable`, pri čemu su pojedini redovi i ćelije dostupni kroz `XWPFTableRow` i `XWPFTableCell`. Za ekstrakciju cjelokupnog teksta dokumenta u jednom koraku dostupna je klasa `XWPFWordExtractor`. Zaglavlja i podnožja dostupna su kroz `XWPFHeader` i `XWPFHeaderFooterPolicy` klase putem `XWPFHeaderFooterPolicy` objekta [6].

3.3. PowerPoint prezentacije

Za rad s PowerPoint prezentacijama Apache POI nudi dvije implementacije. `HSLF` (*Horrible Slide Layout Format*) pokriva stariji binarni `.ppt` format koji odgovara PowerPoint verzijama 97–2003, a `XSLF` (*XML Slide Layout Format*) namijenjen je modernom `.pptx` formatu temeljenom na OOXML standardu. Za razliku od Excel modula koji dijele zajednički SS API, `HSLF` i `XSLF` trenutno nemaju zajedničko sučelje, što znači da kod napisan za jedan format nije izravno prenosiv na drugi [7].

Glavna ulazna točka za rad s `.pptx` datotekama je klasa `XMLSlideShow`, dok je za stariji `.ppt` format to `HSLFSlideShow`. Prezentacija se sastoji od hijerarhijske strukture: prezentacija sadržava slajdove, svaki slajd sadržava oblike (`shapes`), a svaki oblik može sadržavati tekstualne okvire i odlomke. Slajdovi se kreiraju metodom `createSlide()`, pri čemu je moguće odabrati željeni predložak rasporeda (*slide layout*) iz dostupnih rasporeda slide mastera. Oblici poput tekstualnih okvira, slika i tablica dodaju se metodama poput `createTextBox()` i `addPicture()`. Za naprednije slučajeve korištenja, `XSLFSlide` objekt implementira metodu `draw()` koja omogućuje iscrtavanje cijelog slajda na Java `Graphics2D` objekt, što je korisno za generiranje slika ili PDF-ova iz prezentacija [7].

4. Praktični primjeri

Za demonstraciju praktične primjene Apache POI biblioteke koristi se Eclipse IDE uz Maven kao alat za upravljanje ovisnostima. Maven omogućuje jednostavno upravljanje vanjskim bibliotekama kroz centralnu konfiguracijsku datoteku `pom.xml`, čime se izbjegava ručno preuzimanje i dodavanje JAR datoteka u classpath projekta [8].

Nakon kreiranja novog Maven projekta u Eclipseu putem `File → New → Maven Project`, u datoteku `pom.xml` potrebno je dodati dvije ovisnosti. Ovisnost `poi` pokriva podršku za starije binarne formate te predstavlja jezgrenu biblioteku projekta, dok ovisnost `poi-ooxml` dodaje podršku za moderne OOXML formate. Maven automatski preuzima sve tranzitivne ovisnosti, pa nije potrebno ručno upravljati dodatnim JAR datotekama. Trenutno aktualna stabilna verzija Apache POI biblioteke je 5.5.1. [8]

```
1 <dependency>
2     <groupId>org.apache.poi</groupId>
3     <artifactId>poi</artifactId>
4     <version>5.5.1</version>
5 </dependency>
6 <dependency>
7     <groupId>org.apache.poi</groupId>
8     <artifactId>poi-ooxml</artifactId>
9     <version>5.5.1</version>
10 </dependency>
```

Isječak koda 1: Ovisnosti u datoteci `pom.xml`

4.1. Kreiranje Word dokumenta

Kreiranje Word dokumenta u Apache POI-u temelji se na klasi `XWPFDocument`, koja predstavlja glavni objekt dokumenta. Iz nje se kreiraju svi ostali elementi — zaglavlja, podnožja, paragrafi i tablice [9].

4.1.1. Zaglavlje i podnožje

Zaglavlje i podnožje kreiraju se metodama `createHeader()` i `createFooter()` uz parametar `HeaderFooterType.DEFAULT`, koji označava da se primjenjuju na sve stranice dokumenta. Unutar zaglavlja i podnožja kreira se paragraf, a tekst se dodaje kroz `XWPFRun` objekt. Isti princip — paragraf koji sadrži jedan ili više `XWPFRun` objekata — koristi se za sve tekstualne elemente u dokumentu [9].

```
1 XWPFHeader zaglavlje = dokument.createHeader(HeaderFooterType.DEFAULT);
2 XWPFParagraph zaglavljeParagraf = zaglavlje.createParagraph();
3 zaglavljeParagraf.setAlignment(ParagraphAlignment.CENTER);
4 XWPFRun zaglavljeRun = zaglavljeParagraf.createRun();
5 zaglavljeRun.setText("Fakultet organizacije i informatike");
```

```
6 zaglavljeRun.setFontSize(10);
7 zaglavljeRun.setColor("888888");
```

Isječak koda 2: Obrada Word dokumenta

4.1.2. Paragrafi i stilovi

Paragrafi se kreiraju metodom `createParagraph()` na objektu dokumenta. Svaki paragraf može imati postavljeno poravnanje (`setAlignment()`), razmak prije ili nakon (`setSpacingBefore()`, `setSpacingAfter()`) te jedan ili više `XWPFRun` objekata koji nose stvarni tekst. Na razini `XWPFRun` objekta definiraju se stilska svojstva teksta: font (`setFontFamily()`), veličina (`setFontSize()`), boja (`setColor()`), podebljanje (`setBold()`) i kurziv (`setItalic()`). U primjeru su na ovaj način kreirani naslov, podnaslov, uvodni paragraf i završna napomena [9].

```
1 XWPFParagraph naslov = dokument.createParagraph();
2 naslov.setAlignment(ParagraphAlignment.CENTER);
3 naslov.setSpacingAfter(200);
4 XWPFRun naslovRun = naslov.createRun();
5 naslovRun.setText("Popis studenata i ocjena");
6 naslovRun.setBold(true);
7 naslovRun.setFontSize(20);
8 naslovRun.setFontFamily("Calibri");
```

Isječak koda 3: Kreiranje naslova

4.1.3. Tablica

Tablica se kreira metodom `createTable()` uz navođenje broja redova i stupaca. U primjeru je kreirana tablica s 6 redova i 3 stupca — prvi red služi kao zaglavlje tablice, a preostalih 5 redova sadrži podatke o studentima [9].

Zaglavlje tablice stilizira se plavom pozadinskom bojom (`setColor()`) i bijelim podebljanim tekstom. Svaka ćelija zaglavlja dohvaća se metodom `getCell()` iz odgovarajućeg reda, a tekst se dodaje na isti način kao i u paragrafima dokumenta — kroz `XWPFParagraph` i `XWPFRun` objekte [9].

```
1 XWPFTable tablica = dokument.createTable(6, 3);
2 tablica.setWidth("100%");
3
4 String[] zaglavljaStupaca = {"Ime i prezime", "JMBAG", "Ocjena"};
5 XWPFTableRow zaglavljeRed = tablica.getRow(0);
6 for (int i = 0; i < zaglavljaStupaca.length; i++) {
7     XWPFTableCell celija = zaglavljeRed.getCell(i);
8     celija.setColor("2E75B6");
9     XWPFParagraph p = celija.getParagraphs().get(0);
10    p.setAlignment(ParagraphAlignment.CENTER);
```

```

11     XWPFRun r = p.createRun();
12     r.setText(zaglavljajaStupaca[i]);
13     r.setBold(true);
14     r.setColor("FFFFFF");
15     r.setFontSize(11);
16 }

```

Isječak koda 4: Kreiranje zaglavlja tablice

Redovi s podacima pune se iteracijom kroz dvodimenzionalno polje. Radi bolje čitljivosti, primijenjeno je izmjenično bojanje redova — parni redovi ostaju bijeli, a neparni dobivaju svijetloplavu pozadinsku boju [9].

```

1  for (int i = 0; i < podaci.length; i++) {
2      XWPFTableRow red = tablica.getRow(i + 1);
3      String boja = (i % 2 == 0) ? "FFFFFF" : "DEEAF1";
4      for (int j = 0; j < podaci[i].length; j++) {
5          XWPFTableCell celija = red.getCell(j);
6          celija.setColor(boja);
7          XWPFParagraph p = celija.getParagraphs().get(0);
8          p.setAlignment(ParagraphAlignment.CENTER);
9          XWPFRun r = p.createRun();
10         r.setText(podaci[i][j]);
11         r.setFontSize(11);
12     }
13 }

```

Isječak koda 5: Punjenje tablice podacima

4.1.4. Spremanje dokumenta

Po završetku izgradnje dokumenta, korištenjem `FileOutputStream` sadržaj se zapisuje u datoteku objekta. Preporučuje se korištenje `try-with-resources` bloka koji osigurava automatsko zatvaranje toka podataka nakon zapisivanja, čime se sprječava curenje resursa [10].

```

1  try (FileOutputStream out = new FileOutputStream("studenti.docx")) {
2      dokument.write(out);
3  }
4  dokument.close();

```

Isječak koda 6: Spremanje dokumenta

Na Slici 1 prikazan je finalni izgled dokumenta u formatu `.docx` generiranog programskim putem. Vizualni prikaz potvrđuje uspješnu integraciju svih ključnih komponenti Apache POI API-ja o kojima je bilo riječi u prethodnim poglavljima.

U gornjem dijelu dokumenta vidljivo je zaglavlje (*header*) s centriranim tekstom institucije, dok središnjim dijelom dominira naslov definiran povećanim fontom i podebljanim stilom (**bold**),

čime se postiže jasna vizualna hijerarhija. Ispod naslova nalazi se tekstualni odlomak koji služi kao uvod u podatke.

Središnji strukturni element dokumenta je tablica s podacima o studentima. Na slici se jasno razaznaje primjena stilova nad tablicom:

- **Zaglavlje tablice** istaknuto je tamnijom bojom pozadine i bijelim tekstom radi bolje čitljivosti.
- **Tijelo tablice** koristi tehniku izmjeničnog bojanja redova (tzv. *zebra striping*), što olakšava praćenje podataka u redovima.
- **Poravnanje sadržaja** unutar ćelija je uniformno, što demonstrira preciznu kontrolu nad `XWPFTableCell` objektima.

Konačno, na dnu stranice nalazi se završna napomena, a cijeli dokument zadržava standardne margine i tipografiju karakterističnu za moderne Microsoft Word dokumente, iako je u potpunosti generiran bez korištenja samog Microsoft Office sučelja.

Fakultet organizacije i informatike - Varaždin

Popis studenata i ocjena

Akadska godina 2025./2026.

U nastavku se nalazi popis studenata s postignutim ocjenama na kolegiju Napredne WEB tehnologije i servisi. Tablica prikazuje ime i prezime studenta, JMBAG te završnu ocjenu.

Ime i prezime	JMBAG	Ocjena
Emanuel Pračić	0123456789	Izvrstan (5)
Pero Kos	0123456789	Izvrstan (5)
Franjo Kovač	0123456790	Vrlo dobar (4)
Maja Perić	0123456791	Izvrstan (5)
Luka Novak	0123456792	Dobar (3)
Sara Babić	0123456793	Vrlo dobar (4)

Dokument generiran programski korištenjem Apache POI biblioteke.

Slika 1: Word dokument kreiran pomoću Apache POI biblioteke

4.1.5. Izrada cirkularnog dokumenta

Na ovom mjestu će se opisati ideja igradnje cirkularnog pisam uz pomoć `XWPFDocument` klase. Cirkularno pismo je tehnika masovne personalizacije dokumenta. Prije nego što se uopće krene s implementacijom, potrebno je kreirati predložak dokumenta koji sadrži rezervirana mjesta (placeholderi) za dinamičke podatke. U konkretnom primjeru, u Word dokumentu se koriste oznake `{{IME}}`, `{{PREZIME}}`, `{{JMBAG}}`, `{{KOLEGIJ}}`, `{{DATUM}}` i `{{OCJENA}}` koje

će se kasnije zamijeniti stvarnim vrijednostima iz CSV datoteke kao izvora podataka. Ove oznake su lako prepoznatljive i ne sadrže razmake, što olakšava njihovu identifikaciju u kodu.

Predložak bi izgledao na sljedeći način:

Poštovani/a {{IME}} {{PREZIME}},

obavještavamo Vas kako ste na kolegiju {{KOLEGIJ}} ostvarili ocjenu {{OCJENA}}.

JMBAG: {{JMBAG}}

Datum: {{DATUM}}

S poštovanjem,

Fakultet organizacije i informatike

Za potrebu popune navedenog predloška, koristi se CSV datoteka koja sadrži podatke o studentima i njihovim ocjenama. Ova datoteka služi kao izvor podataka za zamjenu rezerviranih mjesta u predlošku, omogućujući generiranje personaliziranih dokumenata za svakog studenta.

```
1 ime, prezime, jmbag, kolegij, ocjena
2 Pero, Kos, 0123456789, Napredne WEB tehnologije i servisi, Izvrstan (5)
3 Franjo, Kovač, 0123456790, Napredne WEB tehnologije i servisi, Vrlo dobar (4)
4 Maja, Perić, 0123456791, Napredne WEB tehnologije i servisi, Izvrstan (5)
5 Luka, Novak, 0123456792, Napredne WEB tehnologije i servisi, Dobar (3)
6 Sara, Babić, 0123456793, Napredne WEB tehnologije i servisi, Vrlo dobar (4)
```

Isječak koda 7: Datoteka studenti.csv

U Prilogu se nalazi Java kod koji demonstrira kako se koristi Apache POI za čitanje predloška, zamjenu rezerviranih mjesta s podacima iz CSV datoteke i generiranje personaliziranih Word dokumenata za svakog studenta. Ovaj primjer ilustrira praktičnu primjenu Apache POI biblioteke u kontekstu masovne personalizacije dokumenata, što je česta potreba u obrazovnim institucijama i poslovnim okruženjima.

Program se sastoji od dvije metode: `main()` koja upravlja ukupnim tokom izvođenja i pomoćne metode `zamijeniUParagrafu()` koja provodi zamjenu placeholdera unutar pojedinog paragrafa.

U metodi `main()` najprije se definiraju putanje do predloška i CSV datoteke te se automatski generira trenutni datum korištenjem `LocalDate.now()`. CSV datoteka čita se redak po redak korištenjem `BufferedReader` klase, pri čemu se prvi redak (zaglavlje) preskače zastavicom `prviRed`. Svaki sljedeći redak dijeli se po zarezu metodom `split(",")`, a vrijednosti se pohranjuju u zasebne varijable. Za svakog studenta predložak se učitava kao novi `XWPFDocument` objekt kako bi svaki generirani dokument krenuo od čistog predloška. Po završetku zamjene, dokument se sprema pod nazivom oblika `obavijest_Ime_Prezime.docx`.

Metoda `zamijeniUParagrafu()` rješava specifičan problem XWPF implementacije — Word dokument interno može razbiti jedan vidljivi tekst u više `XWPFRun` objekata, zbog čega jednostavna zamjena po pojedinom odsječku ne bi mogla prepoznati cijeli placeholder. Metoda stoga najprije spaja tekstove svih odsječaka paragrafa u jedan string korištenjem

`StringBuilder` klase, zatim provodi zamjenu svih placeholdera, a konačni tekst upisuje u prvi odsječak dok ostale odsječke prazni. Na taj način stilska svojstva prvog odsječka ostaju sačuvana, a sadržaj je ispravno popunjen.

4.2. Generiranje Excel izvještaja

Primjer automatskog generiranja tablice i grafa iz podataka.

```
1 % TODO: dodaj Java kod ovdje
```

Isječak koda 8: Generiranje Excel izvještaja

5. Zaključak

Sažetak ključnih spoznaja i smjernice za primjenu Apache POI-a.

Popis isječaka koda

1.	Ovisnosti u datoteci pom.xml	6
2.	Obrada Word dokumenta	6
3.	Kreiranje naslova	7
4.	Kreiranje zaglavlja tablice	7
5.	Punjenje tablice podacima	8
6.	Spremanje dokumenta	8
7.	Datoteka studenti.csv	10
8.	Generiranje Excel izvještaja	11
9.	Cirkularno pismo	18

Popis literature

- [1] Apache Software Foundation, *Apache POI; Project History* — *poi.apache.org*, <https://poi.apache.org/devel/history/index.html>, [Pristupljeno 24. travnja 2026.]
- [2] Apache Software Foundation, *Apache POI - Wikipedia* — *en.wikipedia.org*, https://en.wikipedia.org/wiki/{A}pache_{P}{O}{I}#{H}istory_and_roadmap, [Pristupljeno 24. travnja 2026.]
- [3] Apache Software Foundation, *Apache POI-Component Overview* — *poi.apache.org*, <https://poi.apache.org/components/>, [Pristupljeno 25. travnja 2026.]
- [4] Apache Software Foundation, *Apache POI 5.5.1 API Documentation*, <https://poi.apache.org/apidocs/dev/>, [Pristupljeno 24. travnja 2026.]
- [5] Apache Software Foundation, *POI-HSSF and POI-XSSF/SXSSF - Java API To Access Microsoft Excel Format Files* — *poi.apache.org*, <https://poi.apache.org/components/spreadsheet/>, [Pristupljeno 29. travnja 2026.]
- [6] Apache Software Foundation, *Apache POI-HWPF and XWPF - Java API to Handle Microsoft Word Files*, <https://poi.apache.org/components/document/>, [Pristupljeno 29. travnja 2026.]
- [7] Apache Software Foundation, *Apache POI-HSLF - A Quick Guide* — *poi.apache.org*, <https://poi.apache.org/components/slideshow/quick-guide.html>, [Pristupljeno 29. travnja 2026.]
- [8] Apache Software Foundation, *Apache POI - Download Release Artifacts* — *poi.apache.org*, <https://poi.apache.org/download.html>, [Pristupljeno 05. svibnja 2026.]
- [9] Apache Software Foundation, *XWPF Quick Guide*, <https://poi.apache.org/components/document/quick-guide-xwpf.html>, [Pristupljeno 14. svibnja 2026.]
- [10] Apache Software Foundation, *HWPF and XWPF — Java API to Handle Microsoft Word Files*, <https://poi.apache.org/components/document/>, [Pristupljeno 14. svibnja 2026.]

Popis slika

1.	Word dokument kreiran pomoću Apache POI biblioteke	9
----	--	---

Popis tablica

1. Pregled Apache POI modula	3
--	---

Prilozi

Prilog

Cirkularno pismo

```
1 package com.zpracic.word;
2
3 import java.io.*;
4 import java.time.LocalDate;
5 import java.time.format.DateTimeFormatter;
6 import org.apache.poi.xwpf.usermodel.*;
7
8 public class CirkularnoPismo {
9
10     public static void main(String[] args) throws IOException {
11         String csvDatoteka = "src/main/java/com/zpracic/word/studenti.csv";
12         String predlozakDatoteka = "src/main/java/com/zpracic/word/
13             ↪ predlozak.docx";
14         String datum = LocalDate.now()
15             .format(DateTimeFormatter.ofPattern("dd.MM.yyyy."));
16
17         try (BufferedReader br = new BufferedReader(
18             new FileReader(csvDatoteka))) {
19             String linija;
20             boolean prviRed = true;
21             while ((linija = br.readLine()) != null) {
22                 if (prviRed) {
23                     prviRed = false;
24                     continue;
25                 }
26                 String[] podaci = linija.split(",");
27                 String ime = podaci[0];
28                 String prezime = podaci[1];
29                 String jmbag = podaci[2];
30                 String kolegi = podaci[3];
31                 String ocjena = podaci[4];
32
33                 try (FileInputStream fis = new FileInputStream(
34                     ↪ predlozakDatoteka);
35                     XWPFDocument dokument = new XWPFDocument(fis)) {
```

```

34
35         for (XWPFParagraph paragraf : dokument.getParagraphs())
36             ↪ {
37                 zamijeniUParagrafu(paragraf, ime, prezime,
38                                     jmbag, kolegij, ocjena, datum);
39             }
40
41         String izlazDatoteka = "src/main/java/com/zpracic/word/
42             ↪ obavijest_"
43             + ime + "_" + prezime + ".docx";
44         try (FileOutputStream out =
45             new FileOutputStream(izlazDatoteka)) {
46             dokument.write(out);
47         }
48         System.out.println("Kreiran: " + izlazDatoteka);
49     }
50     System.out.println("Cirkularno pismo uspješno generirano.");
51 }
52
53 private static void zamijeniUParagrafu(XWPFParagraph paragraf,
54     String ime, String prezime, String jmbag,
55     String kolegij, String ocjena, String datum) {
56
57     StringBuilder cijeli = new StringBuilder();
58     for (XWPFRun run : paragraf.getRuns()) {
59         if (run.getText(0) != null) {
60             cijeli.append(run.getText(0));
61         }
62     }
63     String tekst = cijeli.toString();
64     tekst = tekst.replace("{{IME}}", ime);
65     tekst = tekst.replace("{{PREZIME}}", prezime);
66     tekst = tekst.replace("{{JMBAG}}", jmbag);
67     tekst = tekst.replace("{{KOLEGIJ}}", kolegij);
68     tekst = tekst.replace("{{OCJENA}}", ocjena);
69     tekst = tekst.replace("{{DATUM}}", datum);
70
71     if (!paragraf.getRuns().isEmpty()) {
72         paragraf.getRuns().get(0).setText(tekst, 0);
73         for (int i = 1; i < paragraf.getRuns().size(); i++) {
74             paragraf.getRuns().get(i).setText("", 0);
75         }
76     }
77 }
78 }

```